

Gestione File — find, tar, rsync, Backup

Modulo: 2C | **Corso:** Lab Amministrazione di Sistemi T

Prerequisiti: Modulo 2B completato

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Fonte: “Gestione di utenti e file [19 marzo]” (sezione file) + LAB esercizi 3–6

Ricerca nel filesystem — find

`find` esplora il filesystem **in tempo reale** cercando file che soddisfano una combinazione di criteri. È potente ma ha un costo: percorre fisicamente le directory.

Sintassi base

```
find <dove> <criteri> <azione>
```

Criteri principali

Criterio	Esempio	Significato
<code>-name</code>	<code>-name '*.log'</code>	Nome del file (wildcard tra apici)
<code>-type</code>	<code>-type f / -type d</code>	Tipo: <code>f</code> =file, <code>d</code> =directory, <code>l</code> =link
<code>-size</code>	<code>-size +100k</code>	Dimensione: <code>+</code> maggiore di, <code>-</code> minore di
<code>-mtime</code>	<code>-mtime -2</code>	Modificato negli ultimi 2 giorni (<code>-</code> =entro, <code>+</code> =più di)
<code>-user</code>	<code>-user alice</code>	Proprietario specifico
<code>-nouser</code>	<code>-nouser</code>	Nessun utente corrispondente (file “orfani”)
<code>-perm</code>	<code>-perm -2000</code>	Permessi (es. SGID attivo)

```
# Tutti i file .c sotto /usr/src più grandi di 100KB
```

```
find /usr/src -name '*.c' -size +100k -print
```

```
# File modificati nelle ultime 48 ore
```

```
find /var/log -mtime -2 -type f
```

```
# File orfani (utente eliminato) modificati meno di 2 giorni fa
```

```
find / -type f -nouser -mtime -2
```

Eseguire comandi sui file trovati — `-exec`

`-exec` è una delle opzioni più potenti: per ogni file trovato esegue un comando.

```
find /usr/src -name '*.c' -size +100k -exec cat {} \;  
#
```

```
# / fine del comando
# nome del file corrente
```

- {} viene sostituito di volta in volta con il nome del file trovato
- \; segnala la fine del comando da eseguire (il backslash fa l'escape del ;)

```
# Mostrare i permessi di tutti gli script nella home
```

```
find /home -name '*.sh' -exec ls -la {} \;
```

```
# Cercare file con SGID attivo (utile per audit di sicurezza)
```

```
find / -perm -2000 -type f -exec ls -la {} \;
```

locate — ricerca rapida su database

locate non esplora il filesystem: cerca in un **database pre-costruito**. Molto più veloce, ma con un trade-off.

```
locate passwd # istantaneo
```

```
sudo updatedb # aggiorna il database (solitamente schedulato automaticamente)
```

	find	locate
Velocità	Lenta (esplora il filesystem)	Istantanea (cerca nel db)
Aggiornamento	Sempre attuale	Obsoleto tra un updatedb e l'altro
Criteri	Tutti (size, mtime, permessi...)	Solo nome del file
Uso tipico	Ricerche precise e condizionali	Trovare rapidamente dove si trova un file

Identificazione del contenuto — file

In Linux le estensioni dei nomi non hanno significato per il sistema — sono solo etichette leggibili. Il comando **file** identifica il vero tipo del contenuto:

```
file /bin/bash
```

```
# /bin/bash: ELF 64-bit LSB pie executable, x86-64...
```

```
file /etc/passwd
```

```
# /etc/passwd: ASCII text
```

```
file foto.jpg.txt # anche se il nome dice .txt
```

```
# foto.jpg.txt: JPEG image data, JFIF standard
```

file usa tre metodi in sequenza: controlla se il file è vuoto o speciale, poi usa i **magic number** (sequenze di byte caratteristiche di ogni formato), poi analisi euristica per testo e linguaggi.

Trasferimento dati a basso livello — dd

dd legge e scrive byte da qualsiasi file, inclusi i file speciali di device. Non c'è filtraggio — trasferisce esattamente quello che trova.

```
dd if=<sorgente> of=<destinazione> bs=<dimensione_blocco> count=<n_blocchi>
```

Parametro	Significato
if=	Input file (sorgente)
of=	Output file (destinazione)
bs=	Block size — dimensione di ogni blocco
count=	Numero di blocchi da trasferire
skip=	Blocchi da saltare all'inizio della sorgente
seek=	Blocchi da saltare all'inizio della destinazione

Lab esercizio 3 — prenotazione spazio su disco

```
dd if=/dev/zero of=file.out bs=4k count=16k
```

- if=/dev/zero — sorgente infinita di byte zero (file speciale sempre disponibile su Linux)
- bs=4k — blocchi da 4 kilobyte
- count=16k — 16.384 blocchi

Dimensione risultante: $4096 \times 16384 = 67.108.864 \text{ byte} = 64 \text{ MB}$

```
ls -lh file.out
```

```
# -rw-r--r-- 1 vagrant vagrant 64M ... file.out
```

Questa tecnica serve a **riservare spazio su disco** in anticipo — ad esempio per creare un file di swap, un'immagine disco, o garantire che uno spazio sia disponibile prima di un'operazione critica.

File aperti — fuser e lsof

Lab esercizio 4 — differenza tra due versioni di script

```
# v1 - apre e chiude il file a ogni iterazione
```

```
while sleep 1; do
```

```
    dd if=/dev/zero bs=1k count=$(( $(echo $RANDOM | rev | cut -c1) + 1 )) >> output
done
```

```
# v2 - il file viene aperto una sola volta, il redirect è fuori dal loop
```

```
while sleep 1; do
```

```
    dd if=/dev/zero bs=1k count=$(( $(echo $RANDOM | rev | cut -c1) + 1 ))
done >> output
```

La differenza è **dove si trova il redirect >>**:

- **v1**: il >> è dentro il loop — ogni iterazione apre il file, scrive, chiude il file
- **v2**: il >> è fuori dal loop — il file viene aperto una volta sola all'avvio dello script, rimane aperto per tutta la durata

Conseguenza pratica: se rinomini o cancelli il file **output** mentre **v2** è in esecuzione, lo script continua a scrivere sul vecchio inode (il file “esiste ancora” perché è aperto), e non si accorge che il nome è stato cambiato. Con **v1**, al ciclo successivo riapre il file per nome — quindi crea un nuovo **output** se quello vecchio è sparito.

Strumenti per ispezionare file aperti

```
# fuser: mostra i PID che hanno aperto un file
fuser output
```

```
# lsof: mostra tutti i file aperti (o filtrati per file/processo)
lsof output
lsof -p <PID>
```

Questi comandi sono fondamentali per diagnosticare: “perché non riesco a smontare questo filesystem?” (qualcuno ha un file aperto), “chi sta scrivendo in questo log?”, “perché lo spazio non si libera anche dopo aver cancellato il file?” (un processo lo tiene aperto).

Archiviazione — tar

tar (Tape ARchiver) raccoglie più file in un unico archivio conservando tutti i metadati: permessi, ownership, timestamp. Non comprime di per sé.

Comandi principali

Opzione	Azione
-c	Crea un nuovo archivio
-x	Extrae file da un archivio
-t	Elenco (table of contents) senza estrarre
-r	Aggiunge file a un archivio esistente

Sempre necessario: **-f <file>** per specificare il nome dell’archivio (senza, **tar** prova a leggere/scrivere su un nastro fisico).

Opzioni comuni

Opzione	Significato
-v	Verboso — stampa i file processati
-p	Preserva permessi e ownership
-C <dir>	Esegue l’operazione come se ci si trovasse in <dir>
-T <file>	Legge la lista dei file da archiviare da un file di testo

Esempi pratici

```
# Creare un archivio delle home di tutti gli utenti
tar cvpf users.tar /home/*
# Nota: tar rimuove automaticamente lo slash iniziale dai path
# → i path nell'archivio sono relativi (es. home/alice/...)

# Elencare il contenuto senza estrarre
tar tf users.tar

# Estrarre in una directory specifica
tar -C /newdisk -xvpf users.tar
# → ricrea la struttura home/alice/... dentro /newdisk

# Pipeline: copiare una gerarchia di file preservando tutto
tar cvpf - /home/* | tar -C /newdisk -xvpf -
#           ^                               ^
#           - = stdout                       - = stdin
```

Compressione

tar non comprime — si combina con un filtro di compressione separato.

I tre formati più comuni su Linux

Estensione	Comando	Opzione tar
.gz	gzip / gunzip	-z
.bz2	bzip2 / bunzip2	-j
.xz	xz / unxz	-J

xz offre la compressione migliore ma è più lento. gzip è il più veloce. bzip2 è nel mezzo.

```
# Creare archivio compresso
tar czf archivio.tar.gz /home/*      # gzip
tar cjf archivio.tar.bz2 /home/*     # bzip2
tar cJf archivio.tar.xz /home/*      # xz

# Estrarre archivio compresso (tar rileva il formato automaticamente)
tar xf archivio.tar.gz
tar xf archivio.tar.bz2
tar xf archivio.tar.xz

# Compressione come filtro (pipeline)
tar cf - * | xz -c > archivio.tar.xz
```

Alias utili

```
zcat file.gz          # equivale a: gzip -dc file.gz
zgrep pattern file.gz # equivale a: gzip -dc file.gz | grep pattern
```

Copia massiva — rsync

rsync è lo strumento standard per sincronizzare directory, localmente o via rete. Il suo vantaggio principale su cp: trasferisce solo le **differenze**, non ricopiando file già presenti a destinazione.

Sintassi

```
rsync [opzioni] SORGENTE DESTINAZIONE
```

Modalità di trasferimento

```
# Copia locale
rsync -av /home/alice/ /backup/alice/

# Copia via SSH (non richiede demone rsync sul server)
rsync -av /home/alice/ user@host:/backup/alice/
rsync -av user@host:/backup/alice/ /home/alice/

# Copia via protocollo nativo rsync (richiede demone rsyncd)
rsync user@host::srcdir /destinazione
```

Opzioni principali

Opzione	Significato
-a	Archive: equivale a -rlptgoD — ricorsivo, preserva link, permessi, timestamps, owner, group, file speciali
-v	Verbose
-r	Ricorsivo
-u	Salta file più nuovi a destinazione
-c	Confronta per checksum invece che per timestamp+size
-b	Backup dei file sovrascritti
--exclude	Esclude path dalla copia
--dry-run	Simula senza copiare nulla (ottimo per verificare prima)

```
# Sincronizzazione tipica preservando tutto
rsync -av --exclude '.cache' /home/alice/ /backup/alice/

# Verifica cosa verrebbe copiato senza farlo davvero
rsync -av --dry-run /home/ /backup/home/
```

Backup — strategie

Il backup è la copia dei dati su un supporto offline. Va pianificato considerando cosa copiare, quando, dove conservare le copie e per quanto.

Full backup

Copia completa di ogni file. Semplice da ripristinare, ma lento e ingombrante — difficile da eseguire frequentemente.

Incremental backup

Copia solo i file cambiati dall'ultimo backup. Adatto all'esecuzione frequente. Per il ripristino completo servono sia il full che tutti gli incrementali successivi. Può essere a più livelli:

Full

```
Incrementale/0 (rispetto al full)
    Incrementale/1a (rispetto all'incrementale/0)
    Incrementale/1b (rispetto all'incrementale/0)
Incrementale/0b (rispetto al full)
```

Cautele

Rischio	Misura
File aperti/database in scrittura	Pianificare il backup in orari di basso traffico o usare snapshot
Riservatezza dei dati	Il backup contiene tutto — va protetto fisicamente o cifrato
Affidabilità del supporto	Testare periodicamente che i dati siano leggibili
Reperimento	Organizzare le copie in modo da trovare rapidamente quello che serve

Lab esercizi 5 e 6 — script con find

Esercizio 5 — copia “flat”

Obiettivo: copiare in una directory (secondo parametro) tutti i file contenuti in una directory (primo parametro) modificati da meno di N giorni (terzo parametro), senza preservare la struttura delle sottodirectory.

```
#!/bin/bash
# Uso: ./copla_flat.sh <sorgente> <destinazione> <giorni>
```

```
SRC="$1"
DST="$2"
GIORNI="$3"
```

```
# Verifica parametri
if [ -z "$SRC" ] || [ -z "$DST" ] || [ -z "$GIORNI" ]; then
    echo "Uso: $0 <sorgente> <destinazione> <giorni>"
    exit 1
fi

mkdir -p "$DST"

find "$SRC" -maxdepth 1 -type f -mtime "$GIORNI" -exec cp {} "$DST" \;
```

`-maxdepth 1` limita la ricerca alla directory specificata senza scendere nelle sottodirectory (copia “flat” = senza struttura).

Esercizio 6 — copia con conservazione della struttura

Obiettivo: copiare in una directory (primo parametro) tutti i file del sistema modificati da meno di N giorni (secondo parametro) e tutti i file sotto /home appartenenti a UID senza utente valido, mantenendo la struttura originale delle directory.

```
#!/bin/bash
# Uso: ./copia_struttura.sh <destinazione> <giorni>

DST="$1"
GIORNI="$2"

if [ -z "$DST" ] || [ -z "$GIORNI" ]; then
    echo "Uso: $0 <destinazione> <giorni>"
    exit 1
fi

mkdir -p "$DST"

# File recenti ovunque nel sistema
find / -type f -mtime "$GIORNI" -exec cp --parents {} "$DST" \;
```

`cp --parents` preserva il path completo — se copi /var/log/syslog in /backup, il risultato è /backup/var/log/syslog, non /backup/syslog.

```
# File orfani sotto /home
find /home -type f -nouser -exec cp --parents {} "$DST" \;
```

Riepilogo comandi

Comando	Funzione
<code>find /path -name '*.log'</code>	Cerca file per nome
<code>find /path -mtime -2 -type f</code>	File modificati nelle ultime 48h

Comando	Funzione
<code>find /path -nouser</code>	File orfani (utente eliminato)
<code>find /path -exec comando {} \;</code>	Esegue un comando su ogni file trovato
<code>locate nome</code>	Ricerca rapida su database
<code>sudo updatedb</code>	Aggiorna il database di locate
<code>file nome</code>	Identifica il tipo di contenuto di un file
<code>dd if=src of=dst bs=4k count=16k</code>	Copia a basso livello / prenotazione spazio
<code>fuser file</code>	Mostra i processi che hanno il file aperto
<code>lsof file</code>	Dettagli sui file aperti
<code>tar czf archivio.tar.gz /path</code>	Crea archivio compresso con gzip
<code>tar xf archivio.tar.gz</code>	Estrae un archivio
<code>tar tf archivio.tar.gz</code>	Elenca il contenuto senza estrarre
<code>rsync -av src/ dst/</code>	Sincronizza directory preservando tutto
<code>rsync -av --dry-run src/ dst/</code>	Simula rsync senza copiare

Collegati

- [\[\[Appunti_modulo2C\]\]](#)
- [\[\[appunti_modulo2C_file_tar_rsync_backup\]\]](#)

Hub: [\[\[master_map_studio\]\]](#) · [\[\[glossario_sysadm\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[troubleshooting_vm\]\]](#)
 · [\[\[metodo_studio_esami_pratici\]\]](#)